MLOPS PIPELINES WITH INDUSTRY TOOLS

1: Introduction to Tool-Driven MLOps Systems

1.1 Evolution of Machine Learning Systems

Machine learning systems have evolved significantly over the last decade. Early machine learning workflows were often built using standalone scripts written in programming languages such as Python, where individual stages of model development were executed independently. Data preprocessing, feature engineering, model training, evaluation, and prediction were frequently handled through disconnected scripts with limited coordination between components.

Although such workflows are useful for experimentation and learning, they become increasingly difficult to maintain as machine learning systems scale. Modern machine learning applications operate in environments where datasets continuously evolve, models require regular updates, and systems must maintain reliability over long periods of operation. In such situations, manually executed workflows introduce significant challenges related to consistency, reproducibility, and maintenance.

Traditional machine learning systems often face several operational limitations:

- Lack of reproducibility in experiments
- Difficulty in tracking changes to datasets and models
- Limited automation in training and deployment
- Inconsistent behaviour between development and production environments
- Inability to monitor changing data patterns over time

These limitations highlight the need for a more structured approach to managing machine learning systems.

Machine Learning Operations (**MLOps**) emerged as a discipline that combines principles from machine learning, software engineering, and DevOps to support the reliable development, deployment, and maintenance of machine learning systems. Rather than treating model training as an isolated activity, MLOps focuses on managing the complete lifecycle of machine learning systems.

The evolution from manually developed pipelines to structured MLOps systems can be visualised in **Figure 1.1**.

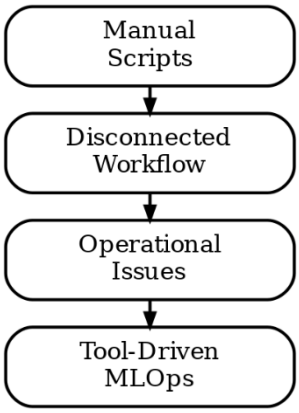

Figure 1.1: Evolution from Manual Machine Learning Pipelines to Tool-Driven MLOps Systems

1.2 Need for Tool-Driven MLOps Systems

Modern machine learning systems involve multiple interconnected activities, including data validation, experimentation, deployment, monitoring, and retraining. Managing these processes manually increases operational complexity and introduces risks related to inconsistency and human error.

To overcome these challenges, organisations increasingly rely on **industry tools** that automate specific stages of the machine learning lifecycle. Instead of building every component manually, specialised tools are integrated into the system to handle well-defined responsibilities.

The adoption of industry tools improves machine learning systems in several ways.

Reproducibility

Machine learning experiments must be reproducible so that results can be verified and repeated under the same conditions. Without reproducibility, model behaviour becomes difficult to trust.

Industry tools help maintain reproducibility by:

- Tracking datasets and their versions
- Recording experiment configurations
- Storing evaluation metrics
- Preserving model artefacts

Reliability

Reliable machine learning systems ensure that incorrect or inconsistent data does not silently affect predictions.

Reliability is improved through:

- Data validation mechanisms
- Standardised preprocessing pipelines
- Structured monitoring processes

Automation

Machine learning workflows involve repetitive activities such as testing, retraining, and deployment. Manual execution of these activities reduces efficiency and increases the likelihood of operational errors.

Automation enables:

- Pipeline execution without manual intervention
- Consistent testing procedures
- Automated deployment workflows
- Repeatable experimentation

Scalability

As machine learning systems grow in complexity, manually maintaining components becomes increasingly inefficient.

Tool-driven systems support scalability through:

- Modular architectures
- Automated workflows
- Reusable pipelines
- Standardised system behaviour

A tool-integrated machine learning lifecycle can be visualised in **Figure 1.2**.

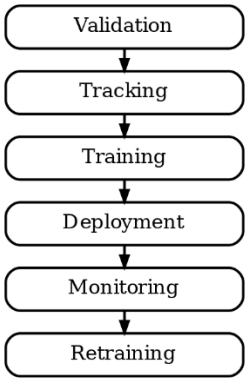

Figure 1.2: Lifecycle of a Tool-Driven MLOps System

1.3 Architecture of Tool-Driven MLOps Systems

Tool-driven MLOps systems can be understood as a set of interconnected layers, where each layer is responsible for a specific stage of the machine learning lifecycle. These layers collectively ensure that machine learning systems remain reproducible, reliable, scalable, and maintainable.

The architecture of a tool-driven MLOps system can be visualised in **Figure 1.3**.

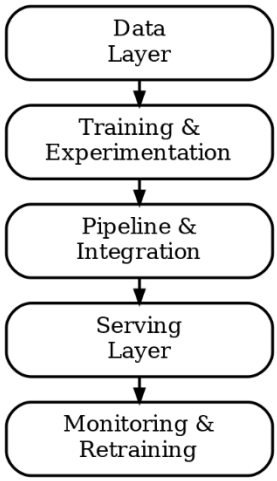

Figure 1.3: Architecture of a Tool-Driven MLOps System

The architecture generally consists of five major layers.

1.3.1 Data Layer

The data layer is responsible for managing both training and production datasets. Since machine learning performance strongly depends on data quality, this layer ensures that datasets are validated, versioned, and consistently maintained.

The major responsibilities of the data layer include:

- Data validation
- Dataset versioning
- Schema enforcement
- Input consistency checks

Without proper data management, machine learning systems may produce unreliable predictions due to inconsistencies between training and production environments.

1.3.2 Training and Experimentation Layer

The training and experimentation layer focuses on model development and performance improvement.

This layer includes activities such as:

- Model selection
- Hyperparameter optimisation
- Experiment tracking
- Model evaluation

Instead of relying on isolated experiments, structured experimentation ensures that different models and configurations can be compared systematically.

Experiment tracking enables teams to analyse:

- Parameters used during training
- Evaluation metrics
- Model artefacts
- Training outcomes

This improves reproducibility and supports informed decision-making during model selection.

1.3.3 Pipeline and Integration Layer

The pipeline layer connects different system components into a unified workflow.

Rather than executing independent scripts manually, pipeline orchestration ensures that machine learning stages execute in a consistent order.

Typical pipeline activities include:

- Data validation
- Feature engineering
- Model training
- Evaluation
- Logging
- Model registration

This layer improves operational consistency and reduces human intervention.

1.3.4 Serving Layer

The serving layer makes trained models available for prediction tasks.

Models may be deployed as:

- Real-time APIs
- Batch inference systems
- Integrated application services

This layer ensures that trained models can generate predictions consistently in production environments.

1.3.5 Monitoring and Retraining Layer

Machine learning systems operate in changing environments where incoming data distributions may evolve over time.

The monitoring layer helps identify:

- Data drift
- Performance degradation
- Feature distribution changes

Monitoring enables organisations to decide when retraining becomes necessary, ensuring that models remain effective under changing conditions.

1.4 Industry Tools Used in MLOps Systems

Different tools are integrated into MLOps systems to manage specific responsibilities within the lifecycle. Instead of using a single platform for all operations, organisations typically combine specialised tools that work together within a structured workflow.

Table 1.1 summarises the major tools commonly used in tool-driven MLOps systems.

MLOps Stage	Tool	Primary Responsibility
Data Validation	Pandera	Schema validation
Data Versioning	DVC	Dataset tracking
Experiment Tracking	MLflow	Metrics and model logging
Hyperparameter Optimisation	Optuna	Model tuning
CI/CD Automation	GitHub Actions	Pipeline automation
Deployment	FastAPI + Docker	Model serving
Monitoring	Evidently	Drift detection

Table 1.1: Tool Mapping Across the MLOps Lifecycle

The relationship between tools and lifecycle stages can be visualised in **Figure 1.4**.

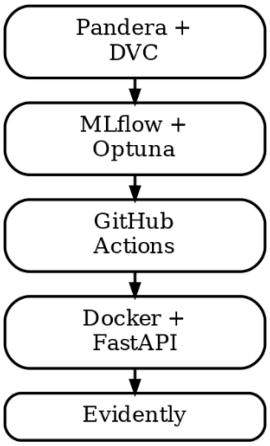

Figure 1.4: Tool Mapping Across the MLOps Lifecycle

1.5 Benefits of Tool-Integrated MLOps Systems

The integration of industry tools transforms machine learning workflows into structured systems capable of operating at scale.

Key benefits include:

- Improved reproducibility
- Better experiment management
- Reliable deployment environments
- Enhanced monitoring capabilities

- Automated workflows
- Improved maintainability

A key insight is that industry tools do not replace machine learning principles. Instead, they operationalise those principles through automation, standardisation, and system integration.

2: Building Reliable Data Systems

2.1 Limitations of Traditional Machine Learning Systems

Traditional machine learning systems are often developed as standalone workflows where datasets are collected, transformed, and used to train models through manually executed scripts. Although this approach may produce acceptable performance during experimentation, it introduces significant limitations when deployed in real-world environments.

A major challenge in traditional systems is the absence of structured validation mechanisms. Incoming datasets are often assumed to be correct without verifying whether they follow expected schemas or constraints. Changes in column names, missing values, incorrect data types, or inconsistent feature distributions may silently affect model behaviour and degrade performance.

Another important limitation is the absence of reproducibility. Since datasets and experiments are not systematically tracked, reproducing previous results becomes difficult. Even minor changes in input data may result in different outcomes, making experimentation unreliable.

Traditional systems also struggle with:

- Inconsistent preprocessing between training and inference
- Lack of experiment tracking
- Limited model traceability
- No monitoring of changing data patterns
- Absence of retraining strategies

As machine learning systems scale, these limitations reduce system reliability and increase operational risk.

The transition from traditional workflows to reliable MLOps systems can be visualised in **Figure 2.1**.

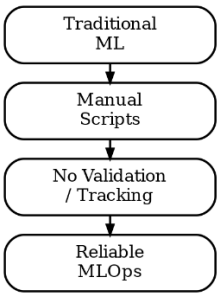

Figure 2.1: Traditional Machine Learning Workflow versus Reliable MLOps Workflow

2.2 Data Reliability in MLOps Systems

Reliable machine learning systems depend heavily on the quality and consistency of data. Since models learn patterns directly from data, errors in datasets can significantly affect prediction quality.

In MLOps systems, data reliability is achieved through structured mechanisms that validate incoming datasets and maintain reproducibility across experiments.

Reliable data systems focus on:

- Validation of incoming data
- Consistency between training and production datasets
- Dataset version control
- Traceability across experiments

Two important tools commonly used for this purpose are **Pandera** and **DVC**.

Pandera ensures that datasets follow predefined schemas before entering the pipeline, while DVC maintains version control for datasets and pipeline artefacts.

Together, these tools ensure that machine learning systems operate on reliable and reproducible data.

2.3 Data Validation using Pandera

Data validation refers to the process of checking whether datasets satisfy predefined requirements before being used in machine learning pipelines.

In many systems, invalid data may silently enter the workflow and cause unexpected failures. Examples include:

- Missing feature values
- Incorrect column types

- Out-of-range values
- Schema mismatches

Pandera provides a structured approach for validating datasets using schema definitions.

A schema defines:

- Expected column names
- Data types
- Value constraints
- Null handling rules

For example, a customer dataset may require age values to remain within a valid range and customer identifiers to follow consistent formats.

Pandera enables:

- Detection of incorrect inputs before training
- Consistent validation during inference
- Reduced risk of silent failures
- Improved reliability of downstream components

The validation process can be visualised in **Figure 2.2**.

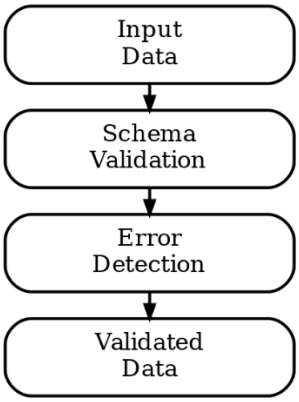

Figure 2.2: Data Validation Workflow using Pandera

2.4 Data Versioning using DVC

Machine learning systems frequently operate on evolving datasets. New records may be introduced, existing features may change, and historical data may be updated over time.

Without proper version control, identifying which dataset version was used during training becomes difficult.

Data Version Control (DVC) provides structured dataset tracking similar to how Git manages source code.

DVC enables:

- Version tracking for datasets
- Reproducibility of experiments
- Restoration of previous data versions
- Consistent collaboration across teams

Each dataset version can be linked to a specific experiment, ensuring that results remain reproducible.

This becomes especially important when comparing multiple models or investigating unexpected changes in performance.

The data versioning process can be visualised in **Figure 2.3**.

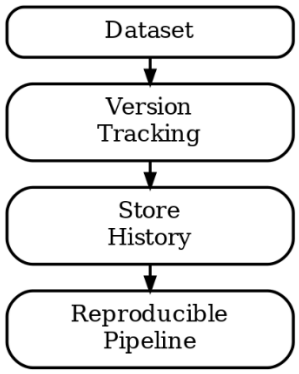

Figure 2.3: Dataset Versioning Workflow using DVC

2.5 Integrated Reliability Workflow

Reliable machine learning systems do not treat validation and versioning as isolated processes. Instead, these mechanisms are integrated directly into the pipeline.

A typical reliability workflow follows several stages:

1. Incoming data is collected
2. Validation rules are applied using Pandera
3. Validated datasets are versioned using DVC

4. Processed data enters the training pipeline
5. Outputs are linked with dataset versions

This integrated approach ensures that machine learning systems remain traceable, reproducible, and resilient to unexpected changes in input data.Reliable data systems form the foundation upon which experimentation, deployment, and monitoring stages are built.

3: Model Development and Experimentation

3.1 Structured Model Development in MLOps

Model development is a central stage of any machine learning system. Traditional workflows often rely on manually selecting algorithms, adjusting parameters through trial and error, and recording results informally. Although such approaches may produce acceptable outcomes for experimentation, they become increasingly difficult to maintain as systems grow in complexity.

In modern MLOps environments, model development is treated as a structured and reproducible process. Instead of relying on isolated experimentation, machine learning workflows integrate systematic evaluation, automated optimisation, and experiment tracking mechanisms.

Structured model development enables organisations to:

- Compare multiple models fairly
- Maintain reproducibility across experiments
- Automate parameter optimisation
- Track model performance consistently
- Preserve trained models for future reuse

This structured approach transforms model development into a repeatable engineering process rather than a collection of disconnected experiments.

The workflow of structured model development can be visualised in **Figure 3.1**.

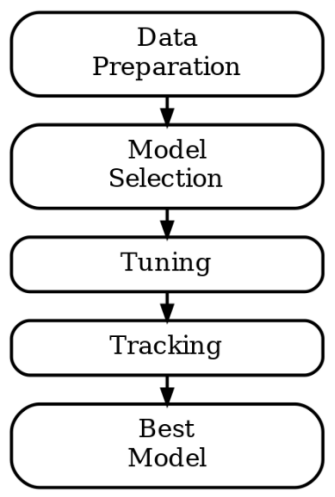

Figure 3.1: Structured Model Development Workflow

3.2 Model Selection in Machine Learning Systems

Selecting an appropriate machine learning model is a critical decision in system development. Different models may produce different outcomes depending on the characteristics of the dataset, feature relationships, and business objectives.

Instead of relying on a single algorithm, modern MLOps systems compare multiple models under consistent evaluation settings.

Common models used in classification tasks include:

- Logistic Regression
- Decision Trees
- Random Forests
- Gradient Boosting Models
- Support Vector Machines

Model comparison becomes reliable only when experiments follow consistent conditions.

A fair comparison generally requires:

- Identical training and testing datasets
- Consistent feature engineering pipelines
- Common evaluation metrics
- Controlled parameter settings

Performance evaluation should align with business objectives. For example, in customer churn prediction systems, identifying high-risk customers may be more important than achieving overall accuracy.

Important evaluation metrics include:

- Accuracy
- Precision
- Recall
- F1-score
- ROC-AUC

The model selection process can be visualised in **Figure 3.2**.

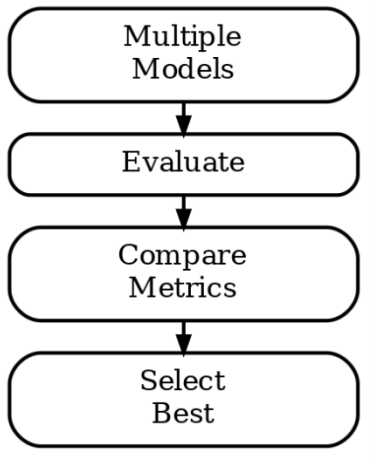

Figure 3.2: Model Selection Workflow

3.3 Hyperparameter Optimisation using Optuna

Machine learning models contain configurable settings known as **hyperparameters**, which influence how models learn from data.

Examples of hyperparameters include:

- Learning rate
- Tree depth
- Number of estimators
- Batch size

Selecting appropriate hyperparameters significantly affects model performance. Traditionally, these parameters were adjusted manually through repeated experimentation, which is both time-consuming and inefficient.

Optuna is an optimisation framework that automates hyperparameter tuning by systematically exploring multiple parameter combinations.

Instead of manually selecting configurations, Optuna:

- Defines a search space
- Evaluates multiple parameter combinations
- Optimises according to a chosen objective metric
- Selects the best-performing configuration

The optimisation process continues until suitable parameter combinations are identified.

Benefits of automated hyperparameter optimisation include:

- Reduced manual effort
- Improved performance consistency
- Faster experimentation cycles
- Better parameter discovery

The optimisation workflow can be visualised in **Figure 3.3**.

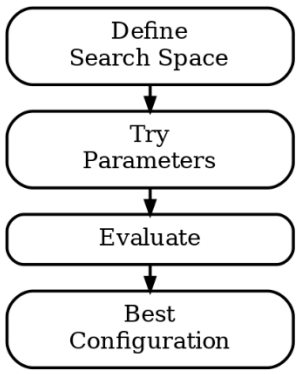

Figure 3.3: Hyperparameter Optimisation Workflow using Optuna

3.4 Experiment Tracking using MLflow

Machine learning experimentation often involves testing multiple models, parameter combinations, and preprocessing techniques. Without systematic tracking, comparing experiments becomes difficult and results may be lost over time.

MLflow is an experiment management platform that records and organises machine learning runs.

MLflow enables systematic tracking of:

- Training parameters
- Evaluation metrics
- Model artefacts
- Experiment configurations

Each experiment run is stored with associated metadata, making it easier to compare outcomes and reproduce previous results.

MLflow improves experimentation by:

- Maintaining structured records of experiments
- Enabling comparisons across runs
- Preserving trained models
- Supporting reproducibility

For example, if multiple tuning experiments are conducted, MLflow enables direct comparison of performance metrics to identify the most effective configuration.

The experiment tracking process can be visualised in **Figure 3.4**.

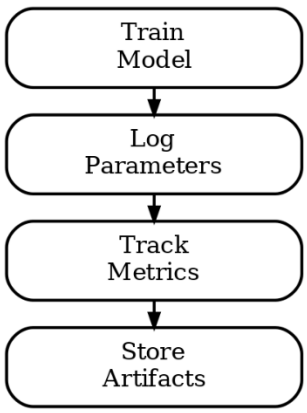

Figure 3.4: Experiment Tracking Workflow using MLflow

3.5 Model Registry and Reproducibility

After model training, organisations often manage multiple versions of trained models.

Some models may still be under experimentation, while others are ready for deployment. Managing these stages manually increases complexity and creates confusion regarding which model should be used in production.

MLflow provides a **Model Registry** that organises trained models according to lifecycle stages.

Common model stages include:

- Candidate
- Staging
- Production
- Archived

This structured lifecycle management enables teams to:

- Track model versions
- Maintain deployment consistency
- Roll back to earlier models if necessary
- Ensure reproducibility

Reproducibility becomes possible because trained models remain linked to:

- Dataset versions
- Hyperparameters
- Experiment logs
- Evaluation metrics

The model lifecycle can be visualised in **Figure 3.5**.

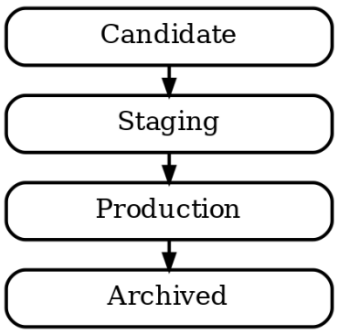

Figure 3.5: MLflow Model Registry Lifecycle

3.6 Integrated Experimentation Workflow

Reliable model development emerges when model selection, optimisation, and tracking operate as interconnected stages rather than isolated activities.

A structured experimentation workflow generally follows these steps:

1. Multiple candidate models are selected
2. Hyperparameter tuning is performed using Optuna
3. Performance is evaluated using consistent metrics
4. Results are logged through MLflow
5. The best-performing model is registered

This integrated approach ensures that model development becomes systematic, reproducible, and scalable. A key insight is that successful machine learning systems depend not only on model accuracy, but also on the ability to reliably reproduce and manage experimentation processes.

4: Production Pipelines, Automation and Deployment

4.1 Production Pipelines in MLOps

Machine learning systems involve multiple stages that extend beyond model development. Before a model becomes suitable for real-world use, datasets must be validated, transformations must be applied consistently, models must be evaluated, and deployment mechanisms must be prepared. Managing these activities manually introduces inconsistency and reduces operational efficiency.

In MLOps environments, these activities are organised into **production pipelines**. A production pipeline refers to a structured workflow where different stages of machine learning execution occur in a predefined order.

Rather than executing independent scripts separately, pipeline orchestration ensures that each stage is connected systematically. Outputs generated during one stage become inputs for the next stage, improving consistency across the system.

A production pipeline generally includes the following stages:

1. Data ingestion and validation
2. Feature engineering and preprocessing
3. Model training

4. Performance evaluation
5. Experiment tracking
6. Model registration
7. Deployment preparation

This integrated structure improves reproducibility and minimises operational errors.

The structure of a production MLOps pipeline can be visualised in **Figure 4.1**.

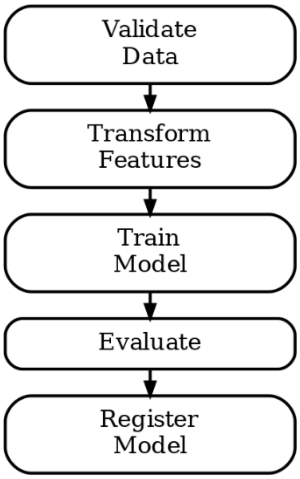

Figure 4.1: End-to-End Production Pipeline Workflow

4.2 Pipeline Integration and Workflow Consistency

A major objective of production pipelines is maintaining consistency between training and deployment environments.

In traditional workflows, preprocessing logic used during model training may differ from production inference systems. Such inconsistencies frequently lead to unreliable predictions and degraded system performance.

Integrated pipelines help reduce this problem by ensuring that:

- Validation rules remain consistent
- Feature transformations are reusable
- Model configurations are traceable
- Evaluation standards remain stable

Workflow consistency improves trust in deployed systems because outputs remain predictable under repeated execution.

Production pipelines also reduce dependency on manual intervention, enabling machine learning systems to scale efficiently.

4.3 Continuous Integration and Continuous Deployment (CI/CD)

Modern machine learning systems evolve continuously. New datasets become available, model improvements are introduced, and deployment environments require updates.

Manually handling these updates increases operational complexity and introduces inconsistencies. Therefore, MLOps systems commonly rely on **Continuous Integration and Continuous Deployment (CI/CD)** practices.

Continuous Integration (CI) refers to the process of automatically validating updates whenever changes are introduced into the system.

This typically includes:

- Dependency checks
- Validation tests
- Pipeline verification
- Model evaluation

Continuous Deployment (CD) focuses on automatically releasing validated models into production environments.

Together, CI/CD ensures that:

- Workflows remain consistent
- Deployment risks are reduced
- Errors are identified earlier
- System maintenance becomes easier

The CI/CD lifecycle can be visualised in **Figure 4.2**.

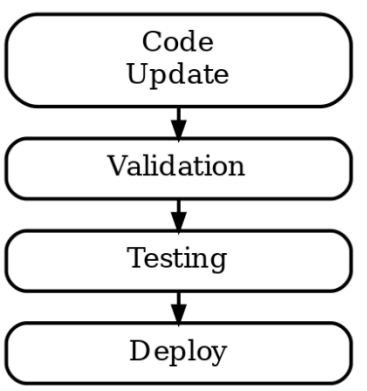

Figure 4.2: CI/CD Workflow in MLOps Systems

4.4 Workflow Automation using GitHub Actions

Automation plays a critical role in maintaining reliable machine learning systems. Instead of manually executing workflows after every update, automation platforms can trigger pipeline execution whenever code changes occur.

GitHub Actions is a workflow automation platform integrated into GitHub repositories.

GitHub Actions enables automated execution of:

- Validation workflows
- Model training pipelines
- Evaluation procedures
- Deployment activities

Workflow instructions are generally written using **YAML configuration files**, where execution steps are explicitly defined.

A typical automated workflow may include:

1. Triggering execution after code commits
2. Installing required dependencies
3. Running validation checks
4. Executing training scripts
5. Evaluating model performance
6. Registering approved models

Automation reduces repetitive manual effort while improving reproducibility and consistency.

The automation workflow can be visualised in **Figure 4.3**.

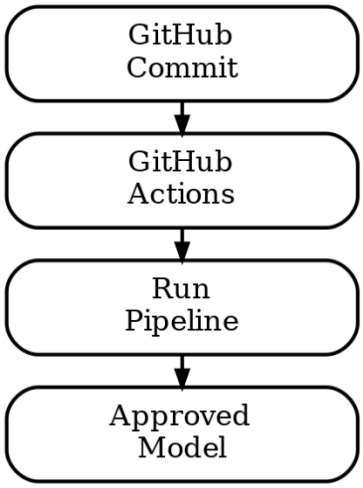

Figure 4.3: GitHub Actions Workflow for MLOps Automation

4.5 Model Deployment using FastAPI

After model development is completed, trained models must become accessible for prediction tasks.

Deployment enables machine learning systems to generate predictions using incoming data.

A common deployment approach in MLOps systems involves exposing trained models as **Application Programming Interfaces (APIs)**.

FastAPI is a lightweight Python framework commonly used for serving machine learning models.

FastAPI enables:

- Real-time inference
- API-based communication
- Structured request handling
- Scalable deployment

In real-world systems, users or applications send requests containing input data, and the deployed model returns predictions.

For example, in a customer churn system, user data may be passed into an API endpoint, where the trained model predicts whether a customer is likely to leave a service.

FastAPI improves deployment reliability because prediction logic remains centralised and reusable.

4.6 Containerisation using Docker

Deploying machine learning systems across different environments often creates compatibility challenges. A system functioning correctly during development may fail after deployment due to dependency mismatches or environmental inconsistencies.

To solve this problem, machine learning applications are commonly packaged using **Docker**.

Docker enables applications and dependencies to be bundled into isolated containers.

These containers include:

- Model files
- Dependencies
- Configuration settings
- Prediction logic

Containerisation improves:

- Portability
- Deployment consistency
- Dependency management
- Scalability

By packaging all components together, Docker ensures that systems behave consistently across environments.

The deployment workflow involving Docker and FastAPI can be visualised in **Figure 4.4**.

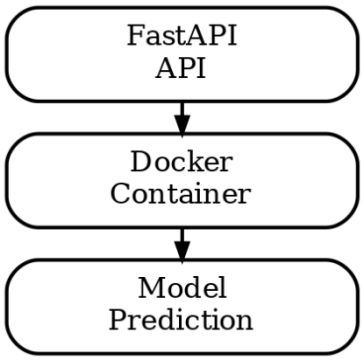

Figure 4.4: Docker and FastAPI Deployment Workflow

4.7 Importance of Automated Deployment Pipelines

Reliable deployment systems reduce operational risk and improve scalability.

Automated deployment pipelines enable organisations to:

- Deploy models faster
- Reduce human error
- Maintain reproducibility
- Standardise workflows

As machine learning systems continue to evolve, production pipelines, automation, and deployment become essential for maintaining long-term reliability.

5: Monitoring, Drift Detection and Retraining

5.1 Monitoring Machine Learning Systems

The deployment of a machine learning model does not mark the end of the machine learning lifecycle. Unlike traditional software systems, machine learning systems operate in dynamic environments where incoming data continuously changes. As a result, model performance may gradually deteriorate even when no changes are made to the model itself.

A model that performs effectively during training may become unreliable in production environments due to changing customer behaviour, evolving business conditions, or shifts in feature distributions.

For this reason, **continuous monitoring** becomes an essential component of MLOps systems.

Monitoring enables organisations to evaluate whether deployed models continue to behave as expected after deployment.

Common monitoring activities include:

- Tracking prediction quality
- Monitoring feature distributions
- Measuring inference latency
- Recording system logs
- Detecting unusual model behaviour

Without proper monitoring, organisations may remain unaware of performance degradation until business outcomes are negatively affected.

The monitoring lifecycle can be visualised in **Figure 5.1**.

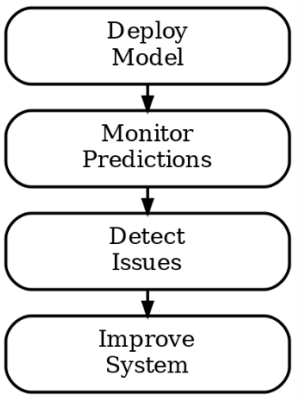

Figure 5.1: Monitoring Lifecycle in Machine Learning Systems

5.2 Data Drift in Machine Learning Systems

Machine learning systems assume that incoming production data follows patterns similar to training data. However, in real-world environments, this assumption may not always hold true.

Over time, customer behaviour, operational conditions, and external influences may alter the characteristics of incoming data.

This phenomenon is commonly referred to as **data drift**.

Data drift occurs when the statistical distribution of production data changes relative to training data.

Examples include:

- Changes in customer purchasing behaviour
- Variations in transaction frequencies
- Shifts in seasonal trends
- New behavioural patterns emerging over time

Even relatively small changes in feature distributions can influence prediction quality.

If drift remains undetected, models may continue generating inaccurate predictions despite previously demonstrating strong performance.

5.3 Drift Detection using Evidently

To detect changes in production data, MLOps systems often rely on monitoring tools capable of analysing distribution shifts.

Evidently is an open-source monitoring framework designed to support drift detection and performance monitoring in machine learning systems.

Evidently compares historical training data against incoming production data to identify statistical deviations.

The framework supports:

- Feature distribution comparison
- Statistical drift reports
- Model performance analysis
- Visual monitoring dashboards

A typical drift detection workflow includes:

1. Selecting a reference dataset
2. Collecting incoming production data
3. Comparing statistical distributions
4. Detecting feature-level changes
5. Generating monitoring reports

Drift detection enables organisations to identify problems before model performance deteriorates significantly.

The Evidently workflow can be visualised in **Figure 5.2**.

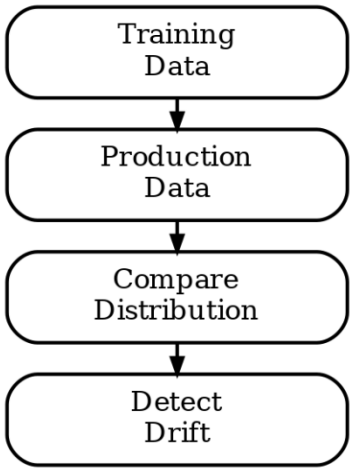

Figure 5.2: Drift Detection Workflow using Evidently

5.4 Observability in MLOps Systems

In addition to monitoring prediction quality, machine learning systems require visibility into operational behaviour.

This capability is known as **observability**.

Observability refers to the ability to understand how machine learning systems behave internally through logs, metrics, and system outputs.

Observability enables teams to analyse:

- Prediction outputs
- Response latency
- System failures
- Experiment history
- Resource utilisation

Logs generated during deployment provide important insights into model behaviour. Important forms of logging include:

- **Prediction Logs:** These logs record model predictions and associated input information.
- **Latency Logs:** Latency logs measure response time during inference.
- **Experiment Logs:** Experiment logs maintain historical information about training runs and model versions.

Strong observability improves debugging, system transparency, and operational reliability.

The observability workflow can be visualised in **Figure 5.3**.

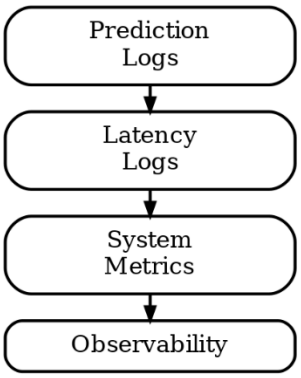

Figure 5.3: Observability Pipeline in MLOps Systems

5.5 Batch Inference Systems

Machine learning systems commonly generate predictions through **real-time inference**, where predictions are produced immediately after receiving input data.

However, some business scenarios require predictions for large datasets simultaneously rather than individual requests.

This process is referred to as **batch inference**.

Batch inference enables:

- Large-scale prediction generation
- Scheduled processing of datasets
- Efficient resource utilisation

Examples include:

- Monthly customer churn prediction
- Fraud analysis across transaction records
- Large-scale recommendation generation

Compared with real-time inference, batch inference prioritises efficiency and scalability over response speed.

5.6 Retraining Strategies for Machine Learning Systems

Monitoring and drift detection help determine when retraining becomes necessary.

Retraining ensures that machine learning systems remain aligned with evolving data distributions.

Retraining may be triggered by:

- Significant data drift
- Degraded model performance
- Scheduled update cycles

Several retraining approaches are commonly used.

- **Fine-Tuning:** Fine-tuning updates an existing model using recently collected data. This method is computationally efficient because the model builds upon existing knowledge rather than restarting training from scratch.

- **Mixed Training:** Mixed training combines historical data with recent production data. This approach reduces the risk of forgetting previously learned patterns while adapting to new trends.

- **Re-Experimentation:** In situations where substantial drift occurs, organisations may repeat the full experimentation process, including:
  - Model selection
  - Hyperparameter tuning
  - Experiment tracking
  - Evaluation

Although computationally expensive, re-experimentation may produce more reliable outcomes under significant environmental changes.

The retraining decision process can be visualised in **Figure 5.4**.

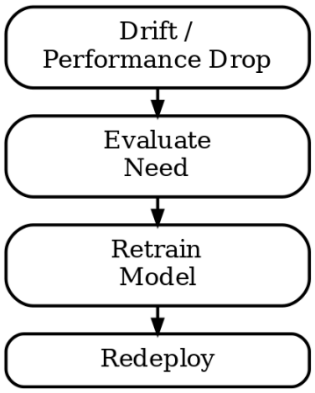

Figure 5.4: Retraining Decision Workflow

5.7 Importance of Monitoring and Retraining

Machine learning systems must be treated as continuously evolving systems rather than static deployments.

Monitoring, drift detection, observability, and retraining collectively ensure that machine learning models remain:

- Reliable
- Reproducible
- Accurate
- Adaptable

Through continuous monitoring and systematic retraining, organisations can maintain long-term performance even in changing environments.